



Τμήμα Εφαρμοσμένης Πληροφορικής
Εργασία στο μάθημα Γλώσσες Προγραμματισμού και
Μεταγλωττιστές

“BISON Συντακτική και Σημασιολογική Ανάλυση”

Τσιμπίνη Πετρούλα ics23085

Μάιος 2026

1.Εισαγωγή

Η παρούσα εργασία αφορά την υλοποίηση ενός συντακτικού και σημασιολογικού αναλυτή για μια γλώσσα μοντελοποίησης προβλημάτων περιορισμών, με την χρήση των εργαλείων Flex για λεκτική ανάλυση και Bison για συντακτική και σημασιολογική ανάλυση.

Το πρόγραμμα αποτελείται από δύο αρχεία :

- constraints_lexer.l :Ο λεκτικός αναλυτής (Flex)
- constraints.y :Ο συντακτικός/σημασιολογικός αναλυτής (Bison)

2.Λεκτική Ανάλυση – Flex(constraints_lexer.l)

2.1 Κανονικοί Ορισμοί

Ορίζονται οι ακόλουθοι βασικοί κανονικοί ορισμοί για την αναγνώριση λεκτικών μονάδων:

```
%{  
#include <stdio.h>  
#include <string.h>  
#include <stdlib.h>  
#include "types.h"  
%}  
  
%option yylineno  
  
ws      [ \t]+  
newline \n|\x0A|\x0D\x0A  
D       [0-9]  
LETTER  [a-zA-Z]  
ID       {LETTER}({LETTER}|{D}|_)*  
INTEGER {D}+  
FLOAT   {D}+\.{D}+
```

Χρησιμοποιούμε την εντολή `newline \n|\x0A|\x0D\x0A` όπως δίνεται από τις οδηγίες της εκφώνησης για να καλύψουμε και τις δύο συμβάσεις τερματισμού γραμμής (Unix/Windows). Επιπλέον η εντολή `FLOAT {D}+\.{D}+` επιτρέπει την δημιουργία πραγματικών αριθμών της μορφής 0.9 ή 12.0 αλλά δεν επιτρέπει την δημιουργία .9 ή 12. .Τέλος, η επιλογή `%option yylineno` ενεργοποιεί την αυτόματη παρακολούθηση αριθμού γραμμής από το Flex.

2.2 Κανόνες Αναγνώρισης

Οι κανόνες αναγνώρισης χωρίζονται σε πέντε κατηγορίες.

Κενά και αλλαγές γραμμής. Τα κενά, τα tabs και οι αλλαγές γραμμής αγνοούνται πλήρως δηλαδή δεν επιστρέφεται κανένα token στον αναλυτή.

```
/*Αγνοούμε τα κενά και τις αλλαγές γραμμών δεν επιστρέφουμε κάποιο token */  
{ws}      { /* do nothing */ }  
{newline} { /* do nothing */ }
```

Keywords. Τα reserved words της γλώσσας αναγνωρίζονται ως strings και επιστρέφουν το αντίστοιχο token. Στην κατηγορία αυτή ανήκουν οι λέξεις δήλωσης μεταβλητών (var, par, set_of), οι βασικοί τύποι (bool, int, float), οι λέξεις περιορισμών και επίλυσης (constraint, solve, satisfy, maximize), οι λογικές σταθερές (true, false), οι λογικοί τελεστές (and, or) και οι τελεστές συνόλων (intersect, union, diff, subset, in).

```
"var"      { return T_VAR; }  
"par"      { return T_PAR; }  
"set_of"   { return T_SET_OF; }  
"bool"     { yyval.se.type = type_bool;    return T_BASE_TYPE; }  
"int"      { yyval.se.type = type_integer; return T_BASE_TYPE; }  
"float"    { yyval.se.type = type_float;   return T_BASE_TYPE; }  
"constraint" { return T_CONSTRAINT; }  
"solve"    { return T_SOLVE; }  
"satisfy"  { return T_SATISFY; }  
"maximize" { return T_MAXIMIZE; }  
"true"     { return T_TRUE; }  
"false"    { return T_FALSE; }  
"and"      { return T_AND; }  
"or"       { return T_OR; }  
"intersect" { return T_INTERSECT; }  
"union"    { return T_UNION; }  
"diff"     { return T_DIFF; }  
"subset"   { return T_SUBSET; }  
"in"       { return T_IN; }
```

Βασικοί τύποι. Τα keywords bool, int και float πέρα από το token T_BASE_TYPE που επιστρέφουν, ενημερώνουν και τον Bison για τον συγκεκριμένο τύπο μέσω του πεδίου yyval.se.type. Αυτό επιτρέπει στον αναλυτή να διακρίνει αμέσως αν πρόκειται για int, float ή bool χωρίς επιπλέον έλεγχο.

```

    { return T_SET_OR; }
    "bool"      { yylval.se.type = type_bool;    return T_BASE_TYPE; }
    "int"       { yylval.se.type = type_integer; return T_BASE_TYPE; }
    "float"     { yylval.se.type = type_float;   return T_BASE_TYPE; }
    "constraint" { return T_CONSTRAINT; }

```

Τελεστές και σύμβολα. Οι σύνθετοι τελεστές ($\Rightarrow$, $\geq$, $\leq$ κλπ) αναγνωρίζονται ως literal strings και επιστρέφουν τα αντίστοιχα named tokens. Οι απλοί χαρακτήρες ($>$, $<$, $=$, $+$, $-$, $*$, $/$, $:$, $:$, $:$) επιστρέφονται απευθείας ως χαρακτήρες στον αναλυτή.

```

"==>" { return T_IMPLIES; }
">="  { return T_GE; }
"<="  { return T_LE; }
".."  { return T_DOTDOT; }
">"   { return '>'; }
"<"   { return '<'; }
"="   { return '='; }
"+"   { return '+'; }
"-"   { return '-'; }
"*"   { return '*'; }
"/"   { return '/'; }
":"   { return ':'; }
";"   { return ';'; }

```

Αριθμοί και αναγνωριστικά. Για τους αριθμούς, ο κανόνας του FLOAT τοποθετείται πριν από τον INTEGER ώστε ένας αριθμός όπως 3.14 να μην αναγνωριστεί εκ λάθους ως ακέραιος 3 ακολουθούμενος από τελεία. Και για τους δύο τύπους αριθμών, αποθηκεύεται το κείμενο του token μέσω strdup(yytext) στο πεδίο info, ενώ το πεδίο numtype λαμβάνει τιμή 0 για ακραίους και 1 για πραγματικούς το οποίο χρησιμοποιείται στον έλεγχο ορίων Range_type. Για τα αναγνωριστικά, αποθηκεύεται το κείμενό τους στο yylval.id με strdup(yytext).

```

/* Βασικοί τύποι δεδομένων: Πέρα από το token T_BASE_TYPE, ενημερώνουμε τον Bison
   για τον συγκεκριμένο τύπο μέσω της δομής yylval (πεδίο type) */
{FLOAT} { yylval.se.info = strdup(yytext); yylval.se.numtype = 1; return T_NUM; }
{INTEGER} { yylval.se.info = strdup(yytext); yylval.se.numtype = 0; return T_NUM; }
{ID}      { yylval.id = strdup(yytext); return T_ID; }

```

Μη έγκυροι χαρακτήρες. Τέλος, ο κανόνας με την τελεία (.) ταιριάζει με οποιονδήποτε χαρακτήρα που δεν καλύφθηκε από τους παραπάνω κανόνες ($\$, \#$) και τον αγνοεί, αποτρέποντας την εκτύπωσή του στην έξοδο.

```

/*Αν βρεθεί οποιοσδήποτε άλλος μη έγκυρος χαρακτήρας (π.χ $, #),
τον αγνοούμε ώστε να μην εκτυπωθεί στην έξοδο όπως ορίζει η εκφώνηση. */
. { /* Do nothing */ }
|
%%

```

3. Συντακτική Ανάλυση – Bison (constraints.y)

3.1 Δομή του αρχείου

Το αρχείο διακρίνεται σε 3 βασικά μέρη:

1. Τμήμα δηλώσεων: ορισμός tokens, τύπος union, προτεραιότητας και προσεταιριστικότητας τελεστών.
2. Τμήμα κανόνων γραμματικής: ένας κανόνας για κάθε μη τερματικό σύμβολο με ενσωματωμένες σημασιολογικές πράξεις.
3. Τμήμα κώδικα C: συναρτήσεις yyerror() και main().

Στο τμήμα δηλώσεων ορίζονται τα ακόλουθα tokens: T_VAR, T_PAR, T_SET_OF, T_CONSTRAINT, T_SOLVE, T_SATISFY, T_MAXIMIZE, T_TRUE, T_FALSE για τα keywords, T_BASE_TYPE για τους βασικούς τύπους (bool, int, float) που μεταφέρουν και τον τύπο μέσω yyval.se.type, T_NUM για αριθμούς (ακέραιους και πραγματικούς), T_ID για αναγνωριστικά, T_DOTDOT για το .., T_IMPL για το ==>, T_GE, T_LE για τους τελεστές >=, <= και T_INTERSECT, T_UNION, T_DIFF, T_SUBSET, T_IN για τους τελεστές συνόλων.

3.2 Προτεραιότητα και Προσεταιριστικότητα Τελεστών

Στο Bison, όσο πιο κάτω ορίζουμε έναν τελεστή τόσο μεγαλύτερη προτεραιότητα έχει.

```

/* Ορισμός προτεραιότητας και προσεταιριστικότητας τελεστών με βάση τις οδηγίες από την
εκφώνηση*/
%nonassoc T_IMPLIES
%left T_OR
%left T_AND
%nonassoc '>' '<' '=' T_GE T_LE
%left '+' '-'
%left '*' '/'

```

Οι εντολές που ξεκινούν με %nonassoc σημαίνει πως είναι μη προσεταιριστικοί ενώ οι %left σημαίνει πως είναι αριστερά προσεταιριστικοί.

3.3 Ανάκτηση από Σφάλματα

Ο κανόνας error ';' Items στα non-terminal Items επιτρέπει στον αναλυτή να συνεχίσει μετά από συντακτικό σφάλμα, προσπερνώντας μέχρι το επόμενο ';' . Έτσι εντοπίζονται πολλαπλά σφάλματα σε ένα πέρασμα.

```

Items : Item ';' Items
      | error ';' Items /* Ανάνηψη από συντακτικό σφάλμα: προσπερνά το λάθος μέχρι το
επόμενο ';' */
      | /* empty*/ /*Τερματισμός της αναδρομής (κενό) */
      ;

```

4. Σημασιολογική Ανάλυση

4.1 Δομή Union και Τύποι

Για τη μεταφορά πληροφοριών τύπου μεταξύ κανόνων χρησιμοποιείται η %union:

```

/* Η ένωση ορίζει τους τύπους δεδομένων που μπορούν να μεταφέρουν
τα tokens και τα non-terminals*/

%union {
    char *id;
    struct {
        char *info; /*κείμενο του token (π.χ. "3.14") */
        ParType type; /* τύπος της έκφρασης */
        int numtype; /* 0=ακέραιος, 1=πραγματικός */
    } se;
}

```

4.2 Βοηθητικές Συναρτήσεις

Με την συνάρτηση norm(t) κανονικοποιούμε τον τύπο par με το αντίστοιχο var. Για παράδειγμα type_par_int με το type integer.

Η συνάρτηση is_numeric(t) επιστρέφει αν ο τύπος είναι ακέραιος ή πραγματικός (var ή par).

Η συνάρτηση is_set(t) επιστρέφει true αν ο τύπος είναι σύνολο ακεραίων ή πραγματικών.

Η συνάρτηση set_elem(t) επιστρέφει τον τύπο των στοιχείων ενός συνόλου.

Η συνάρτηση typeDefinition(a,b) ελέγχει αν δύο τύποι είναι συμβατοί για αριθμητικές πράξεις.

```

/*Συνάρτηση η οποία κανονικοποιεί τους τύπους των παραμέτρων (par) στους αντίστοιχους
τύπους (var).Αυτό μας βολεύει στους ελέγχους καθώς μια ακέραια παράμετρος συμπεριφέρεται
σημασιολογικά όπως μια ακέραια μεταβλητή.
*/
static ParType norm(ParType t) {
    if (t== type_par_int)      return type_integer;
    if (t==type_par_float)     return type_float;
    if (t==type_par_int_set)   return type_int_set;
    if (t==type_par_float_set) return type_float_set;
    return t;
}

/*Ελέγχει αν ένας τύπος είναι αριθμητικός (int ή float, var ή par) */
static int is_numeric(ParType t) {
    return (t==type_integer || t==type_float ||
            t==type_par_int || t==type_par_float);
}

/*Ελέγχει αν ένας τύπος είναι σύνολο (int set ή float set, var ή par) */
static int is_set(ParType t) {
    return (t==type_int_set || t==type_float_set ||
            t==type_par_int_set || t==type_par_float_set);
}

/*Επιστρέφει τον τύπο των στοιχείων που περιέχονται σε ένα σύνολο*/
static ParType set_elem(ParType t) {
    if (t==type_int_set || t==type_par_int_set) return type_integer;
    if (t==type_float_set || t==type_par_float_set) return type_float;
    return type_error;
}

/*Έλεγχος τύπων για αριθμητικές πράξεις.Επειδή η γλώσσα έχει αυστηρό σύστημα τύπων και
απαγορεύει τις αυτόματες μετατροπές,από την εκφώνηση μας αναφέρατε ότι επιτρέπουμε πράξεις
μόνο μεταξύ ίδιων τύπων*/
ParType typeDefinition(ParType a, ParType b) {
    a= norm(a); b =norm(b);
    if (a==type_integer && b==type_integer) return type_integer;
    if (a==type_float && b==type_float) return type_float;
    return type_error;
}

```

4.3 Πίνακας Συμβόλων

Χρησιμοποιείται η υλοποίηση που παρέχεται από το αρχείο που μας δίνεται από το eclass (symbolTable.h). Κύριες συναρτήσεις:

1. Η addvar(&symbolTable, name, type) κάνει εισαγωγή νέας μεταβλητής. Επιστρέφει false αν υπάρχει ήδη.
2. Η lookup(symbolTable, name) κάνει έλεγχο για την ύπαρξη μεταβλητής.
3. Η lookup_type(symbolTable, name) εκτελεί επιστροφή τύπου μεταβλητής.

4.4 Πίνακας Σημασιολογικών Ελέγχων

Ο αναλυτής υλοποιεί αρκετούς σημασιολογικούς ελέγχους, οι οποίοι ενεργοποιούνται μέσω του YYERROR και εκτυπώνουν το αντίστοιχο μήνυμα στο stderr.

Έλεγχος διπλής δήλωσης μεταβλητής. Κάθε φορά που ορίζεται μια νέα μεταβλητή καλείται η addvar().Αν η συνάρτηση επιστρέψει false, σημαίνει ότι η μεταβλητή υπάρχει ήδη στον πίνακα συμβόλων και εκτυπώνεται Var X

Already Declared!. Αντίστοιχα, σε κάθε χρήση αναγνωριστικού (κανόνες Arg, BoolArg, SolutionType) ελέγχεται πρώτα η ύπαρξή του μέσω lookup() και αν απουσιάζει εκτυπώνεται Undefined Variable X.

```
VarPar_item
/*Δήλωση απλής μεταβλητής απόφασης π.χ var int: x; ή var set_a: newX; */
: "var" Type ':' T_ID
{
    ParType vtype= $2.type;
    /* Αν το Type επιστρέψει type_error αλλά έχει όνομα ($2.info), σημαίνει ότι
    χρησιμοποιήθηκε ένα T_ID ως τύπος (ένα par set που ορίζει τύπο var).
    */
    if (vtype== type_error && $2.info != NULL) {
        ParType pt = lookup_type(symbolTable, $2.info);
        if (pt== type_par_int_set) vtype= type_int_set;
        else if (pt== type_par_float_set) vtype= type_float_set;
        else {
            fprintf(stderr, "Wrong type specifier (line %d) %s\n",
                yylineno, $2.info);
            YYERROR;
        }
    }
}
/*Προσπάθεια εισαγωγής στον πίνακα συμβόλων. Αν αποτύχει, η μεταβλητή έχει ήδη δηλωθεί*/
if (!addvar(&symbolTable, $4, vtype)) {
    fprintf(stderr, "Var %s Already Declared!\n", $4);
    YYERROR;
}
}
```

Έλεγχος χρήσης αδήλωτης μεταβλητής. Ο έλεγχος αυτός γίνεται σε τρία σημεία όπου εμφανίζεται T_ID ως χρήση: στον κανόνα Arg, στον κανόνα BoolArg, και στον κανόνα Type όταν το ID χρησιμοποιείται ως τύπος.

```
/* Αν το όρισμα είναι μεταβλητή (T_ID) ελέγχουμε αν υπάρχει στον πίνακα συμβόλων */
| T_ID
{
    if (!lookup(symbolTable, $1)) {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
    $$ .type= lookup_type(symbolTable, $1);
    $$ .info = $1; $$ .numtype = 0;
}
;
```



```

BoolArg
/*Σύγκριση αριθμητικών εκφράσεων (π.χ x > y) */
: NumExpression RelOp NumExpression
{
    ParType l = norm($1.type), r = norm($3.type);
    /*πρέπει και τα δύο μέλη να είναι αριθμητικά και του ίδιου τύπου*/
    if (!is_numeric($1.type) || !is_numeric($3.type) || l != r) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool;
}
| SetExpression { $$ = $1; }
| "true" { $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0; }
| "false" { $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0; }
| T_ID
{
    /* Έλεγχος αν ένα ID που χρησιμοποιείται αυτούσιο ως λογικό όρισμα είναι όντως
    δηλωμένο ως bool*/
    if (!lookup(symbolTable, $1)) {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
}

```

```

Type
: Base_type { $$ = $1; }
| Range_type { $$ = $1; }
| T_ID
{
    /* Όταν ένα T_ID χρησιμοποιείται ως τύπος, πρέπει να ελέγξουμε αν έχει δηλωθεί
    προηγουμένως ως παράμετρος συνόλου (par set), αλλιώς έχουμε σημασιολογικό
    λάθος */
    if (lookup(symbolTable, $1)) {
        ParType pt = lookup_type(symbolTable, $1);
        if (pt != type_par_int_set && pt != type_par_float_set) {
            fprintf(stderr, "Wrong type specifier (line %d) %s\n", yylineno, $1);
            YYERROR;
        }
    }
    else {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
}

```

Έλεγχος για λανθασμένο τύπο ως specifier για var. Εμφανίζεται σε δύο περιπτώσεις: α) στον κανόνα Type ->T_ID όταν το ID υπάρχει στον πίνακα αλλά δεν είναι type_par_int_set ή type_par_float_set και β) στον κανόνα "var" Type ':' T_ID όταν το \$2.info δείχνει σε ID που δεν έχει αποδεκτό τύπο.

```

type
: Base_type { $$ = $1; }
| Range_type { $$ = $1; }
| T_ID
{
/* Όταν ένα T_ID χρησιμοποιείται ως τύπος, πρέπει να ελέγξουμε αν έχει δηλωθεί
προηγουμένως ως παράμετρος συνόλου (par set), αλλιώς έχουμε σημασιολογικό
λάθος */
if (lookup(symbolTable, $1)) {
ParType pt = lookup_type(symbolTable, $1);
if (pt != type_par_int_set && pt != type_par_float_set) {
fprintf(stderr, "Wrong type specifier (line %d) %s\n", yylineno, $1);
YYERROR;
}
} else {
fprintf(stderr, "Undefined Variable %s\n", $1);
YYERROR;
}
}
}
;

```

```

VarPar_item
/*Δήλωση απλής μεταβλητής απόφασης π.χ var int: x; ή var set_a: newX; */
: "var" Type ':' T_ID
{
ParType vtype= $2.type;
/* Αν το Type επιστρέψει type_error αλλά έχει όνομα ($2.info), σημαίνει ότι
χρησιμοποιήθηκε ένα T_ID ως τύπος (ένα par set που ορίζει τύπο var).
*/
if (vtype== type_error && $2.info != NULL) {
ParType pt = lookup_type(symbolTable, $2.info);
if (pt== type_par_int_set) vtype= type_int_set;
else if (pt== type_par_float_set) vtype= type_float_set;
else {
fprintf(stderr, "Wrong type specifier (line %d) %s\n",
yylineno, $2.info);
YYERROR;
}
}
}

```

Έλεγχος για λανθασμένο τύπο για set_of ή par. Εμφανίζεται σε τρεις περιπτώσεις: α) var set_of bool β) var set_of με ID που δεν αντιστοιχεί σε έγκυρο τύπο συνόλου και γ) par bool (ο τύπος bool δεν υποστηρίζεται για par). Στην περίπτωση γ) αν και δεν πρόκειται για δήλωση συνόλου, ο κώδικας εκτυπώνει το ίδιο μήνυμα Wrong Type in set declaration καθώς το bool δεν είναι έγκυρος τύπος για παράμετρο, οι μόνοι έγκυροι τύποι par είναι int και float.

```

/*Δήλωση μεταβλητής συνόλου π.χ var set_of int : setX; */
| "var" "set_of" Type ':' T_ID
{
    ParType vtype;
    ParType inner= $3.type;
    /* Ανάλογα με τον εσωτερικό τύπο, καθορίζουμε αν είναι σύνολο ακεραίων ή
πραγματικών */
    if (inner ==type_integer)    vtype= type_int_set;
    else if (inner ==type_float) vtype= type_float_set;
    else if (inner ==type_bool) {
        fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
        YYERROR;
    }
}

```

```

/*Δήλωση μεταβλητής συνόλου π.χ var set_of int : setX; */
| "var" "set_of" Type ':' T_ID
{
    ParType vtype;
    ParType inner= $3.type;
    /* Ανάλογα με τον εσωτερικό τύπο, καθορίζουμε αν είναι σύνολο ακεραίων ή
πραγματικών */
    if (inner ==type_integer)    vtype= type_int_set;
    else if (inner ==type_float) vtype= type_float_set;
    else if (inner ==type_bool) {
        fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
        YYERROR;
    } else {
        /* Διαχείριση περίπτωσης όπου χρησιμοποιείται alias (T_ID) μέσα στο set_of */
        if ($3.info !=NULL) {
            ParType pt= lookup_type(symbolTable, $3.info);
            if (pt ==type_par_int_set)    vtype= type_int_set;
            else if (pt== type_par_float_set) vtype= type_float_set;
            else {
                fprintf(stderr, "Wrong type specifier (line %d) %s\n",
                    yylineno, $3.info);
                YYERROR;
            }
        } else {
            fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
            YYERROR;
        }
    }
}

```

```

/*Δήλωση απλής παραμέτρου π.χ. par int: x; */
| "par" Base_type ':' T_ID
{
    ParType ptype;
    if ($2.type== type_integer)    ptype = type_par_int;
    else if ($2.type== type_float) ptype = type_par_float;
    else {
        /* Το 'par bool' δεν υποστηρίζεται από τη γλώσσα βάσει του Πίνακα 1 */
        fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
        YYERROR;
    }
}

```

Έλεγχος για ανάμειξη τύπων σε αριθμητικές πράξεις. Σε κάθε αριθμητική πράξη (+, -, *, /) καλείται η `typeDefinition()`, η οποία εσωτερικά κανονικοποιεί τους τύπους μέσω `norm()` και ελέγχει αν είναι και οι δύο `type_integer` ή και οι δύο `type_float`. Σε κάθε άλλη περίπτωση (π.χ `int + float`) επιστρέφεται `type_error` και εκτυπώνεται `Type mismatch in line N`. Η ίδια λογική ισχύει και για τις σχεσιακές συγκρίσεις (`RelOp >, <, =, >=, <=`). Στον κανόνα `BoolArg: NumExpression RelOp NumExpression`, ελέγχεται αν και οι δύο παραστάσεις είναι αριθμητικές (`is_numeric()`) και του ιδίου κανονικοποιημένου τύπου μέσω

norm().Αν όχι, εκτυπώνεται Type mismatch in line N.(έχω screenshot σε ακόλουθο έλεγχο)

```
/* -ΑΡΙΘΜΗΤΙΚΕΣ ΕΚΦΡΑΣΕΙΣ- */

NumExpression
: NumExpression '+' NumExpression
{
    $$$.type = typeDefinition($1.type, $3.type);
    if ($$.type == type_error) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
}
```

/*Έλεγχος τύπων για αριθμητικές πράξεις.Επειδή η γλώσσα έχει αυστηρό σύστημα τύπων και απαγορεύει τις αυτόματες μετατροπές,από την εκφώνηση μας αναφέρατε ότι επιτρέπουμε πράξεις μόνο μεταξύ ίδιων τύπων*/

```
ParType typeDefinition(ParType a, ParType b) {
    a= norm(a); b =norm(b);
    if (a==type_integer && b==type_integer) return type_integer;
    if (a==type_float && b==type_float) return type_float;
    return type_error;
}
```

BoolArg

```
/*Σύγκριση αριθμητικών εκφράσεων (π.χ x > y) */
: NumExpression RelOp NumExpression
{
    ParType l = norm($1.type), r = norm($3.type);
    /*πρέπει και τα δύο μέλη να είναι αριθμητικά και του ίδιου τύπου*/
    if (!is_numeric($1.type) || !is_numeric($3.type) || l != r) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$$.type = type_bool;
}
| SetExpression { $$ = $1; }
| "true" { $$$.type = type_bool; $$$.info = NULL; $$$.numtype = 0; }
| "false" { $$$.type = type_bool; $$$.info = NULL; $$$.numtype = 0; }
| T_ID
```

Έλεγχος για λανθασμένα όρια(ανάμειξη int/float).Κατά την ανάλυση ενός T_NUM .. T_NUM,εξετάζεται το πεδίο numtype και των δύο ορίων.Αν ένας είναι αέριαιος (numtype==0) και ο άλλος πραγματικός (numtype==1),εκτυπώνεται Wrong Ranges declared (line N). Should be of same type.

```
/*σφάλμα αν έχουμε ανάμειξη τύπων*/
fprintf(stderr,
    "Wrong Ranges declared (line %d). Should be of same type.\n",
    yylineno);
YYERROR;
}
$$$.info = NULL; $$$.numtype = 0;
}
```

Έλεγχος για ανεστραμμένα ίσα όρια. Όταν και τα δύο όρια είναι όμοια (δηλαδή μόνο ακέραια ή μόνο πραγματικά), ελέγχεται αν το κάτω όριο είναι μικρότερο του άνω. Αν είναι ομοειδή αλλά το κάτω όριο είναι μεγαλύτερο ή ίσο του άνω, εκτυπώνεται Wrong Ranges declared (line N). Lower bound must be less than upper bound.

```
Range_type
: T_NUM ".." T_NUM
{
    /* Έλεγχος αν και τα δύο άκρα είναι ακέραιοι */
    if ($1.numtype == 0 && $3.numtype == 0) {
        $$type = type_integer;
        if (atoi($1.info) >= atoi($3.info)) {
            fprintf(stderr,
                "Wrong Ranges declared (line %d). Lower bound must be less than
upper bound.\n",
                yylineno);
            YYERROR;
        }
    }
    /* Έλεγχος αν και τα δύο άκρα είναι πραγματικοί αριθμοί */
    } else if ($1.numtype == 1 && $3.numtype == 1) {
        $$type = type_float;
        if (atof($1.info) >= atof($3.info)) {
            fprintf(stderr,
                "Wrong Ranges declared (line %d). Lower bound must be less than
upper bound.\n",
                yylineno);
            YYERROR;
        }
    }
}
```

Έλεγχος μη-συνόλων σε σύνολα. Υλοποιούνται τρεις μορφές SetExpression: α) Arg subset Arg Set_Ops Arg (subset με πράξη), β) Arg subset Arg (δυναδική) και γ) Arg in Arg. Και στις τρεις περιπτώσεις ελέγχεται πρώτα αν τα ορίσματα που αναμένονται ως σύνολα είναι όντως σύνολα μέσω is_set(). Στην περίπτωση in, επιπλέον, καλείται η set_elem() για να εξαχθεί ο τύπος των στοιχείων του συνόλου και να συγκριθεί με τον τύπο του πρώτου ορίσματος.

```

SetExpression
/*Σύνθετη περίπτωση subset με πράξη συνόλων( A subset B union C) */
: Arg "subset" Arg Set_Ops Arg
{
    if (!is_set($1.type) || !is_set($3.type) || !is_set($5.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    /* Όλα τα σύνολα στην πράξη πρέπει να είναι του ιδίου τύπου */
    if (norm($1.type)!=norm($3.type) || norm($1.type)!=norm($5.type)) {
        fprintf(stderr, "Type mismatch in sets, line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0;
}

/*Απλό subset μεταξύ δύο συνόλων π.χ A subset B */
| Arg "subset" Arg
{
    if (!is_set($1.type) || !is_set($3.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    if (norm($1.type) != norm($3.type)) {
        fprintf(stderr, "Type mismatch in sets, line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0;
}

/*Έλεγχος αν ένα στοιχείο ανήκει σε ένα σύνολο π.χ x in seta */
| Arg "in" Arg
{
    if (!is_set($3.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
}

```

Έλεγχος ανάμειξης τύπων σε σύνολα. Αφού επαληθευτεί ότι τα ορίσματα είναι σύνολα, ελέγχεται ότι είναι του ίδιου τύπου (π.χ και τα δύο `int_set` ή και τα δύο `float_set`) με χρήση `norm()`.

```

SetExpression
/*Σύνθετη περίπτωση subset με πράξη συνόλων( A subset B union C) */
: Arg "subset" Arg Set_Ops Arg
{
    if (lis_set($1.type) || !lis_set($3.type) || !lis_set($5.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    /* Όλα τα σύνολα στην πράξη πρέπει να είναι του ιδίου τύπου */
    if (norm($1.type)!=norm($3.type) || norm($1.type)!=norm($5.type)) {
        fprintf(stderr, "Type mismatch in sets, line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0;
}

/*Απλό subset μεταξύ δύο συνόλων π.χ A subset B */
| Arg "subset" Arg
{
    if (lis_set($1.type) || !lis_set($3.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    if (norm($1.type) != norm($3.type)) {
        fprintf(stderr, "Type mismatch in sets, line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0;
}

```

Έλεγχος πράξης in(αν τύπος στοιχείου διάφορο από τύπο συνόλου). Μετά τον έλεγχο ότι το δεύτερο Arg είναι σύνολο, ελέγχεται αν ο τύπος του πρώτου Arg (το στοιχείο) ταιριάζει με τον τύπο που περιέχει το σύνολο μέσω set_elem(). π.χ int in float_set θα αποτύχει.

```

/*Έλεγχος αν ένα στοιχείο ανήκει σε ένα σύνολο π.χ x in seta */
| Arg "in" Arg
{
    if (lis_set($3.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    /* Ο τύπος του στοιχείου πρέπει να ταιριάζει με τον τύπο των στοιχείων του
    συνόλου (π.χ int με int_set) */
    if (norm($1.type) != set_elem($3.type)) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0;
}
;

```

Έλεγχοι λογικών εκφράσεων. Στους κανόνες BoolExpr για and, or, ==> και στον κανόνα BoolArg για σχεσιακές συγκρίσεις και για T_ID που χρησιμοποιείται ως λογικό όρισμα, ελέγχεται ότι οι τύποι είναι type_bool. Σε άλλη περίπτωση, εκτυπώνει Type mismatch in line N.

```

/*Λογικές εκφράσεις και έλεγχος τύπων (πρέπει και τα δύο μέλη να είναι boolean) */
BoolExpr
: BoolExpr "==" BoolExpr
{
    if ($1.type != type_bool || $3.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool;
}
| BoolExpr "or" BoolExpr
{
    if ($1.type != type_bool || $3.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool;
}
| BoolExpr "and" BoolExpr
{
    if ($1.type != type_bool || $3.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool;
}
| BoolArg { $$ = $1; }
;

```

```

BoolArg
/*Σύγκριση αριθμητικών εκφράσεων (π.χ x > y) */
: NumExpression RelOp NumExpression
{
    ParType l = norm($1.type), r = norm($3.type);
    /*πρέπει και τα δύο μέλη να είναι αριθμητικά και του ίδιου τύπου*/
    if (!is_numeric($1.type) || !is_numeric($3.type) || l != r) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$ .type = type_bool;
}
| SetExpression { $$ = $1; }
| "true" { $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0; }
| "false" { $$ .type = type_bool; $$ .info = NULL; $$ .numtype = 0; }
| T_ID
{
    /* Έλεγχος αν ένα ID που χρησιμοποιείται αυτούσιο ως λογικό όρισμα είναι όντως
    δηλωμένο ως bool*/
    if (!lookup(symbolTable, $1)) {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
}

```

Έλεγχος maximize. Στον κανόνα SolutionType, αν χρησιμοποιείται το maximize ελέγχεται αν το αναγνωριστικό που ακολουθεί έχει δηλωθεί στον πίνακα συμβόλων. Αν όχι, εκτυπώνεται Undefined Variable X.


```

/* -ΕΝΤΟΛΕΣ ΕΠΙΛΥΣΗΣ- */

Solve_item : "solve" SolutionType ;

SolutionType
    : "satisfy"
    | "maximize" T_ID
    {
        /*στην περίπτωση μεγιστοποίησης, η μεταβλητή πρέπει να έχει δηλωθεί */
        if (!lookup(symbolTable, $2)) {
            fprintf(stderr, "Undefined Variable %s\n", $2);
            YYERROR;
        }
    }
    ;

%%

```

5. Παραδείγματα Εκτέλεσης

Επιτυχής ανάλυση και Συντακτικά σφάλματα

```

petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ make
flex -s -o constraints.lex.c constraints_lexer.l
bison -v -o constraints.tab.c constraints.y
gcc -c -I./symbolTableSrc -I./ symbolTableSrc/symbolTable.c
gcc -o constraints constraints.tab.c symbolTable.o -I./symbolTableSrc -I./ -lfl
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test1
Syntax OK!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test2
Syntax OK!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_error1
Error(line 6) : syntax error, unexpected '>', expecting ';'
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_error2
Error(line 6) : syntax error, unexpected '>', expecting ';'
Error(line 7) : syntax error, unexpected ';', expecting T_NUM or T_ID
Error(line 8) : syntax error, unexpected T_ID
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ |

```

Σημασιολογικά σφάλματα

```

petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./c
onstraints test_semantic_error1
Undefined Variable y
There were 1 errors in code. Failure!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_semantic_error2
Type mismatch in line 5
There were 1 errors in code. Failure!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_semantic_error3
Wrong Ranges declared (line 3). Should be of same type.
Undefined Variable y
There were 2 errors in code. Failure!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_semantic_error4
Var y Already Declared!
Wrong Type in set declaration 7
There were 2 errors in code. Failure!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_semantic_error5
Args are not sets, line 9
Type mismatch in sets, line 10
Type mismatch in line 11
There were 3 errors in code. Failure!
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ ./constraints test_semantic_error6
Wrong type specifier (line 5) seta
petrina@user:/mnt/c/Users/owner/Downloads/coursework1_2026/coursework1_2026$ |

```

6. Κώδικας

6.1 Λεκτικός Αναλυτής – constraints_lexer.l

```
%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "types.h"
%}

%option yylineno

ws      [ \t]+
newline \n|\x0A|\x0D\x0A
D       [0-9]
LETTER  [a-zA-Z]
ID       {LETTER}({LETTER}|{D}|_)*
INTEGER {D}+
FLOAT   {D}+\.{D}+

%%

/*Αγνοούμε τα κενά και τις αλλαγές γραμμών δεν επιστρέφουμε κάποιο token */

{ws}      { /* do nothing */ }
{newline} { /* do nothing */ }

"var"      { return T_VAR; }
"par"      { return T_PAR; }
"set_of"   { return T_SET_OF; }
"bool"     { yylval.se.type = type_bool;   return T_BASE_TYPE; }
"int"      { yylval.se.type = type_integer; return T_BASE_TYPE; }
"float"    { yylval.se.type = type_float;   return T_BASE_TYPE; }
"constraint" { return T_CONSTRAINT; }
"solve"    { return T_SOLVE; }
"satisfy"  { return T_SATISFY; }
"maximize" { return T_MAXIMIZE; }
"true"     { return T_TRUE; }
"false"    { return T_FALSE; }
"and"      { return T_AND; }
"or"       { return T_OR; }
"intersect" { return T_INTERSECT; }
"union"    { return T_UNION; }
"diff"     { return T_DIFF; }
"subset"   { return T_SUBSET; }
```

```

"diff"      { return T_DIFF; }
"subset"    { return T_SUBSET; }
"in"        { return T_IN; }

"==>"      { return T_IMPLIES; }
">="        { return T_GE; }
"<="        { return T_LE; }
".."        { return T_DOTDOT; }
">"         { return '>'; }
"<"         { return '<'; }
"="         { return '='; }
"+"         { return '+'; }
"-"         { return '-'; }
"*"         { return '*'; }
"/"         { return '/'; }
":"         { return ':'; }
";"         { return ';'; }

/* Βασικοί τύποι δεδομένων: Πέρα από το token T_BASE_TYPE, ενημερώνουμε τον Bison
   για τον συγκεκριμένο τύπο μέσω της δομής yyval (πεδίο type) */
{FLOAT}    { yyval.se.info = strdup(yytext); yyval.se.numtype = 1; return T_NUM; }
{INTEGER}   { yyval.se.info = strdup(yytext); yyval.se.numtype = 0; return T_NUM; }
{ID}        { yyval.id = strdup(yytext); return T_ID; }

/*Αν βρεθεί οποιοσδήποτε άλλος μη έγκυρος χαρακτήρας (π.χ $, #),
   τον αγνοούμε ώστε να μην εκτυπωθεί στην έξοδο όπως ορίζει η εκφώνηση. */
. { /* Do nothing */ }

%%

```

6.2 Συντακτικός/Σημασιολογικός Αναλυτής – constraints.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <stdbool.h>

int yylex();
void yyerror (const char * msg);
int yylineno;

#include "types.h" /* Περιέχει τους τύπους της γλώσσας */
#include "symbolTable.h" /* Περιέχει τις συναρτήσεις διαχείρισης του πίνακα συμβόλων */

/*Δείκτης για τον πίνακα συμβόλων*/
ST_TABLE_TYPE symbolTable;

/*Συνάρτηση η οποία κανονικοποιεί τους τύπους των παραμέτρων (par) στους αντίστοιχους τύπους (var).
Αυτό μας βολεύει στους ελέγχους καθώς μια ακέραια παράμετρος συμπεριφέρεται σημασιολογικά όπως μια ακέραια μεταβλητή.
*/
static ParType norm(ParType t) {
    if (t== type_par_int)      return type_integer;
    if (t==type_par_float)    return type_float;
    if (t==type_par_int_set)  return type_int_set;
    if (t==type_par_float_set) return type_float_set;
    return t;
}

/*Ελέγχει αν ένας τύπος είναι αριθμητικός (int ή float, var ή par) */
static int is_numeric(ParType t) {
    return (t==type_integer || t==type_float ||
            t==type_par_int || t==type_par_float);
}

/*Ελέγχει αν ένας τύπος είναι σύνολο (int set ή float set, var ή par) */
static int is_set(ParType t) {
    return (t==type_int_set || t==type_float_set ||
            t==type_par_int_set || t==type_par_float_set);
}

```

```

/*Επιστρέφει τον τύπο των στοιχείων που περιέχονται σε ένα σύνολο*/
static ParType set_elem(ParType t) {
    if (t==type_int_set || t==type_par_int_set) return type_integer;
    if (t==type_float_set || t==type_par_float_set) return type_float;
    return type_error;
}

/*Έλεγχος τύπων για αριθμητικές πράξεις.Επειδή η γλώσσα έχει αυστηρό σύστημα τύπων και απαγορεύει τις αυτόματες μετατροπές,
από την εκφώνηση μας αναφέρατε ότι επιτρέπουμε πράξεις μόνο μεταξύ ίδιων τύπων*/
ParType typeDefinition(ParType a, ParType b) {
    a= norm(a); b =norm(b);
    if (a==type_integer && b==type_integer) return type_integer;
    if (a==type_float && b==type_float) return type_float;
    return type_error;
}

%}

/* Ενεργοποίηση αναλυτικότερων μηνυμάτων σφάλματος από Bison */
#define parse.error verbose

/* Η ένωση ορίζει τους τύπους δεδομένων που μπορούν να μεταφέρουν
τα tokens και τα non-terminals*/

%union {
    char *id;
    struct {
        char *info; /*κείμενο του token (π.χ. "3.14") */
        ParType type; /* τύπος της έκφρασης */
        int numtype; /* 0=ακέραιος, 1=πραγματικός */
    } se;
}

/* Ορισμός των λεκτικών μονάδων και σύνδεσή τους με τα keywords της γλώσσας */
%token T_VAR "var"
%token T_PAR "par"
%token T_SET_OF "set_of"
%token T_CONSTRAINT "constraint"
%token T_SOLVE "solve"
%token T_SATISFY "satisfy"

```

```

%token T_MAXIMIZE "maximize"
%token T_TRUE "true"
%token T_FALSE "false"
%token T_DOTDOT "."
%token T_GE ">="
%token T_LE "<="

%token T_AND "and"
%token T_OR "or"
%token T_IMPLIES "=>"

%token T_INTERSECT "intersect"
%token T_UNION "union"
%token T_DIFF "diff"
%token T_SUBSET "subset"
%token T_IN "in"

/* Σύνδεση των tokens που κουβαλάνε τιμές με τα αντίστοιχα πεδία του union */
%token <se> T_BASE_TYPE
%token <se> T_NUM
%token <id> T_ID

/* Ορισμός προτεραιότητας και προσηταιριστικότητας τελεστών με βάση τις οδηγίες από την εκφώνηση*/
%nonassoc T_IMPLIES
%left T_OR
%left T_AND
%nonassoc '>' '<' '=' T_GE T_LE
%left '+' '-'
%left '*' '/'

/* Καθορισμός του τύπου της δομής union που επιστρέφουν τα non-terminals */
%type <se> Type Base_type Range_type
%type <se> NumExpression Arg
%type <se> BoolExpr BoolArg
%type <se> SetExpression

```

```

/* Αρχικός κανόνας: Αρχικοποιεί τον πίνακα συμβόλων ως κενό και ξεκινά την ανάλυση */
Model : {symbolTable=NULL;} Items
;

Items : Item ';' Items
      | error ';' Items /* Ανάνηψη από συντακτικό σφάλμα: προσπερνά το λάθος μέχρι το επόμενο ';' */
      | /* empty*/ /*Τερματισμός της αναδρομής (κενό) */
      ;

Item : VarPar_item
      | Constraint_item
      | Solve_item
      ;

/* -ΔΗΛΩΣΕΙΣ ΜΕΤΑΒΛΗΤΩΝ ΚΑΙ ΠΑΡΑΜΕΤΡΩΝ- */

VarPar_item
/*Δήλωση απλής μεταβλητής απόφασης π.χ var int: x; ή var set_a: newX; */
: "var" Type ':' T_ID
{
    ParType vtype= $2.type;
    /* Αν το Type επιστρέψει type_error αλλά έχει όνομα ($2.info), σημαίνει ότι
    χρησιμοποιήθηκε ένα T_ID ως τύπος (ένα par set που ορίζει τύπο var).
    */
    if (vtype== type_error && $2.info != NULL) {
        ParType pt = lookup_type(symbolTable, $2.info);
        if (pt== type_par_int_set) vtype= type_int_set;
        else if (pt== type_par_float_set) vtype= type_float_set;
        else {
            fprintf(stderr, "Wrong type specifier (line %d) %s\n",
                yylineno, $2.info);
            YYERROR;
        }
    }
}
/*Προσπάθεια εισαγωγής στον πίνακα συμβόλων. Αν αποτύχει, η μεταβλητή έχει ήδη δηλωθεί*/
if (!addvar(&symbolTable, $4, vtype)) {
    fprintf(stderr, "Var %s Already Declared!\n", $4);
    YYERROR;
}
}

/*Δήλωση μεταβλητής συνόλου π.χ var set_of int : setX; */
| "var" "set_of" Type ':' T_ID
{
    ParType vtype;
    ParType inner= $3.type;
    /* Ανάλογα με τον εσωτερικό τύπο, καθορίζουμε αν είναι σύνολο ακεραίων ή πραγματικών */
    if (inner ==type_integer) vtype= type_int_set;
    else if (inner ==type_float) vtype= type_float_set;
    else if (inner ==type_bool) {
        fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
        YYERROR;
    } else {
        /* Διαχείριση περίπτωσης όπου χρησιμοποιείται alias (T_ID) μέσα στο set_of */
        if ($3.info !=NULL) {
            ParType pt= lookup_type(symbolTable, $3.info);
            if (pt==type_par_int_set) vtype= type_int_set;
            else if (pt== type_par_float_set) vtype= type_float_set;
            else {
                fprintf(stderr, "Wrong type specifier (line %d) %s\n",
                    yylineno, $3.info);
                YYERROR;
            }
        } else {
            fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
            YYERROR;
        }
    }
}
if (!addvar(&symbolTable, $5, vtype)) {
    fprintf(stderr, "Var %s Already Declared!\n", $5);
    YYERROR;
}
}
}

```

```

/*Δήλωση απλής παραμέτρου π.χ. par int: x; */
| "par" Base_type ':' T_ID
{
    ParType ptype;
    if ($2.type== type_integer)    ptype = type_par_int;
    else if ($2.type== type_float) ptype = type_par_float;
    else {
        /* Το 'par bool' δεν υποστηρίζεται από τη γλώσσα βάσει του Πίνακα 1 */
        fprintf(stderr, "Wrong Type in set delcaration %d\n", yylineno);
        YYERROR;
    }
    if (!addvar(&symbolTable, $4, ptype)) {
        fprintf(stderr, "Var %s Already Declared!\n", $4);
        YYERROR;
    }
}

/*Δήλωση παραμέτρου συνόλου με εύρος τιμών π.χ. par set_of 8..20 : set_a; */
| "par" "set_of" Range_type ':' T_ID
{
    ParType ptype;
    if ($3.type== type_integer)    ptype = type_par_int_set;
    else if ($3.type == type_float) ptype = type_par_float_set;
    else { YYERROR; }
    if (!addvar(&symbolTable, $5, ptype)) {
        fprintf(stderr, "Var %s Already Declared!\n", $5);
        YYERROR;
    }
}

;

```

```

Type
: Base_type { $$ = $1; }
| Range_type { $$ = $1; }
| T_ID
{
    /* Όταν ένα T_ID χρησιμοποιείται ως τύπος, πρέπει να ελέγξουμε αν έχει δηλωθεί
    προηγουμένως ως παράμετρος συνόλου (par set), αλλιώς έχουμε σημασιολογικό λάθος */
    if (lookup(symbolTable, $1)) {
        ParType pt = lookup_type(symbolTable, $1);
        if (pt != type_par_int_set && pt != type_par_float_set) {
            fprintf(stderr, "Wrong type specifier (line %d) %s\n", yylineno, $1);
            YYERROR;
        }
    } else {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
    $$ .info    = $1;
    $$ .type    = type_error;
    $$ .numtype = 0;
}

;

Base_type : T_BASE_TYPE { $$ = $1; } ;

/* Κανόνες για εύρος τιμών ( 10..20 ή 1.5..2.5) */
Range_type
: T_NUM ".." T_NUM
{
    /* Έλεγχος αν και τα δύο άκρα είναι ακέραιοι */
    if ($1.numtype == 0 && $3.numtype == 0) {
        $$ .type = type_integer;
        if (atoi($1.info) >= atoi($3.info)) {
            fprintf(stderr,
                "Wrong Ranges declared (line %d). Lower bound must be less than upper bound.\n",
                yylineno);
            YYERROR;
        }
    }
    /* Έλεγχος αν και τα δύο άκρα είναι πραγματικοί αριθμοί */
    if ($1.numtype != 0 && $3.numtype != 0) {
        if (strchr($1.info, '.') && strchr($3.info, '.')) {
            $$ .type = type_float;
        } else {
            $$ .type = type_error;
        }
    }
}

```

```

        if (atoi($1.info) >= atoi($3.info)) {
            fprintf(stderr,
                "Wrong Ranges declared (line %d). Lower bound must be less than upper bound.\n",
                yylineno);
            YYERROR;
        }
        /* Έλεγχος αν και τα δύο άκρα είναι πραγματικοί αριθμοί */
    } else if ($1.numtype == 1 && $3.numtype == 1) {
        $$type = type_float;
        if (atof($1.info) >= atof($3.info)) {
            fprintf(stderr,
                "Wrong Ranges declared (line %d). Lower bound must be less than upper bound.\n",
                yylineno);
            YYERROR;
        }
    } else {
        /* Σφάλμα αν έχουμε ανάμειξη τύπων */
        fprintf(stderr,
            "Wrong Ranges declared (line %d). Should be of same type.\n",
            yylineno);
        YYERROR;
    }
    $$info = NULL; $$numtype = 0;
}

;

/* -ΠΕΡΙΟΡΙΣΜΟΙ ΚΑΙ ΛΟΓΙΚΕΣ ΕΚΦΡΑΣΕΙΣ- */

Constraint_item : "constraint" BoolExpr ;

/*Λογικές εκφράσεις και έλεγχος τύπων (πρέπει και τα δύο μέλη να είναι boolean) */
BoolExpr
: BoolExpr "==" BoolExpr
{
    if ($1.type != type_bool || $3.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$type = type_bool;
}

| BoolExpr "or" BoolExpr
{
    if ($1.type != type_bool || $3.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$type = type_bool;
}

| BoolExpr "and" BoolExpr
{
    if ($1.type != type_bool || $3.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$type = type_bool;
}

| BoolArg { $$ = $1; }
;

BoolArg
/*Σύγκριση αριθμητικών εκφράσεων (π.χ x > y) */
: NumExpression RelOp NumExpression
{
    ParType l = norm($1.type), r = norm($3.type);
    /*πρέπει και τα δύο μέλη να είναι αριθμητικά και του ίδιου τύπου*/
    if (!is_numeric($1.type) || !is_numeric($3.type) || l != r) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$type = type_bool;
}

| SetExpression { $$ = $1; }
| "true" { $$type = type_bool; $$info = NULL; $$numtype = 0; }
| "false" { $$type = type_bool; $$info = NULL; $$numtype = 0; }
| T_ID
{
    /* Έλεγχος αν ένα ID που χρησιμοποιείται αυτούσιο ως λογικό όρισμα είναι όντως δηλωμένο ως bool */
    if (!lookup(symbolTable, $1)) {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
}

```

```

    }
    $$$.type = lookup_type(symbolTable, $1);
    if ($$.type != type_bool) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$$.info = $1; $$$.numtype = 0;
}
;

RelOp : '>' | '<' | '=' | T_GE | T_LE ;

/* -ΑΡΙΘΜΗΤΙΚΕΣ ΕΚΦΡΑΣΕΙΣ- */

NumExpression
: NumExpression '+' NumExpression
{
    $$$.type = typeDefinition($1.type, $3.type);
    if ($$.type == type_error) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
}
| NumExpression '-' NumExpression
{
    $$$.type = typeDefinition($1.type, $3.type);
    if ($$.type == type_error) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
}
| NumExpression '*' NumExpression
{
    $$$.type = typeDefinition($1.type, $3.type);
    if ($$.type == type_error) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
}
| NumExpression '/' NumExpression

```



```

| numexpression / numexpression
{
    $$$.type = typeDefinition($1.type, $3.type);
    if ($$.type == type_error) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
}
| Arg { $$ = $1; }
;

/* -ΕΚΦΡΑΣΕΙΣ ΣΥΝΟΛΩΝ- */
SetExpression
/*Σύνθετη περίπτωση subset με πράξη συνόλων( A subset B union C) */
: Arg "subset" Arg Set_Ops Arg
{
    if (!is_set($1.type) || !is_set($3.type) || !is_set($5.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    /* Όλα τα σύνολα στην πράξη πρέπει να είναι του ιδίου τύπου */
    if (norm($1.type) != norm($3.type) || norm($1.type) != norm($5.type)) {
        fprintf(stderr, "Type mismatch in sets, line %d\n", yylineno);
        YYERROR;
    }
    $$.type = type_bool; $$.info = NULL; $$.numtype = 0;
}

/*Απλό subset μεταξύ δύο συνόλων π.χ A subset B */
| Arg "subset" Arg
{
    if (!is_set($1.type) || !is_set($3.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    if (norm($1.type) != norm($3.type)) {
        fprintf(stderr, "Type mismatch in sets, line %d\n", yylineno);
        YYERROR;
    }
    $$.type = type_bool; $$.info = NULL; $$.numtype = 0;
}

```

```

/*Ελέγχος αν ένα στοιχείο ανήκει σε ένα σύνολο π.χ x in seta */
| Arg "in" Arg
{
    if (!is_set($3.type)) {
        fprintf(stderr, "Args are not sets, line %d\n", yylineno);
        YYERROR;
    }
    /* Ο τύπος του στοιχείου πρέπει να ταιριάζει με τον τύπο των στοιχείων του συνόλου (π.χ int με int_set) */
    if (norm($1.type) != set_elem($3.type)) {
        fprintf(stderr, "Type mismatch in line %d\n", yylineno);
        YYERROR;
    }
    $$$.type = type_bool; $$$.info = NULL; $$$.numtype = 0;
}
;

Set_Ops : "intersect" | "union" | "diff" ;

Arg
/*Αν το όρισμα είναι αριθμητικό literal (T_NUM) */
: T_NUM
{
    $$$.info= $1.info;
    $$$.numtype= $1.numtype;
    $$$.type= ($1.numtype == 0) ? type_integer : type_float;
}

/* Αν το όρισμα είναι μεταβλητή (T_ID) ελέγχουμε αν υπάρχει στον πίνακα συμβόλων */
| T_ID
{
    if (!lookup(symbolTable, $1)) {
        fprintf(stderr, "Undefined Variable %s\n", $1);
        YYERROR;
    }
    $$$.type= lookup_type(symbolTable, $1);
    $$$.info = $1; $$$.numtype = 0;
}
;

```

```

Solve_item : "solve" SolutionType ;

SolutionType
: "satisfy"
| "maximize" T_ID
{
    /*Στην περίπτωση μεγιστοποίησης, η μεταβλητή πρέπει να έχει δηλωθεί */
    if (!lookup(symbolTable,$2)) {
        fprintf(stderr,"Undefined Variable %s\n", $2);
        YYERROR;
    }
}
;

%%
/*Ενσωμάτωση του κώδικα C που παράγει ο Flex για τη λεκτική ανάλυση */
#include "constraints.lex.c"

/*Συνάρτηση διαχείρισης συντακτικών λαθών */
void yyerror (const char * msg)
{
    fprintf(stderr,"Error(line %d) : %s\n", yylineno, msg);
}

int main(int argc, char **argv) {
    ++argv, --argc; /* Προσπέραση του ονόματος του εκτελέσιμου προγράμματος */
    if ( argc > 0 )
        yyin = fopen( argv[0], "r" );
    else
        yyin = stdin;

    int result = yyparse(); /* Εκκίνηση του συντακτικού αναλυτή */

    /*Αν η ανάλυση ολοκληρώθηκε επιτυχώς χωρίς κανένα συντακτικό ή σημασιολογικό λάθος */
    if (result == 0 && yynerrs == 0)
        printf("Syntax OK!\n");
    else
        printf("There were %d errors in code. Failure!\n", yynerrs);
    return result;
}

```